COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***INFORMATION SECURITY LAB***
Instructor	***Ms. Ambreen Gul***
Assignment No.	***04***

Student Details

Registration No.	***Name***
SP24-BSE-045	***SHEIKH FARAZ ADIL***

Task 1: Simple RSA Implementation (Without Libraries) [5]

Objective: Understand the math behind RSA Tasks:

- **Manually implement key generation using small primes**
- **Write functions for:**

o Encrypting a message (using public key)

o Decrypting a message (using private key)

- **Test it on your name or a short string**

CODE:

```
# ---------------- RSA IMPLEMENTATION ----------------

# Step 1: Choose two small prime numbers
p = 3
q = 11

# Step 2: Calculate n
n = p * q   # n = 33

# Step 3: Calculate φ(n)
phi = (p - 1) * (q - 1)   # phi = 20

# Step 4: Choose e (must be coprime with phi)
e = 3   # gcd(3,20)=1 so valid

# Step 5: Find d (modular inverse of e)
# (e * d) % phi = 1
d = 7   # because (3*7) % 20 = 1

print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))
```

```python
# ---------------- ENCRYPTION ----------------
def encrypt(message):  1 usage
    encrypted = []

    for ch in message:
        num = ord(ch)                   # convert character → number
        cipher = (num ** e) % n         # encryption formula
        encrypted.append(cipher)

    return encrypted

# ---------------- DECRYPTION ----------------
def decrypt(cipher_list):  1 usage
    decrypted = ""

    for num in cipher_list:
        plain = (num ** d) % n          # decryption formula
        decrypted += chr(plain)         # convert number → character

    return decrypted

# ---------------- TEST ----------------
message = "HI"   # you can use your name also

print("\nOriginal Message:", message)
```

```python
enc = encrypt(message)
print("Encrypted:", enc)

dec = decrypt(enc)
print("Decrypted:", dec)
```

OUTPUT:

```
C:\Users\hashi\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:\Users
Public Key (e, n): (3, 33)
Private Key (d, n): (7, 33)

Original Message: HI
Encrypted: [18, 13]
Decrypted: ▯▯

Process finished with exit code 0
```

LINE-BY-LINE EXPLANATION:

1. PRIME NUMBERS

p = 3
q = 11

- We choose two small prime numbers.
- These are used to create keys.

2. CALCULATE n

n = p * q

- Multiply both primes.
- This gives n = 33.
- It is used in encryption and decryption.

3. CALCULATE ϕ(n)

phi = (p - 1) * (q - 1)

- Formula gives ϕ(n) = 20.
- It helps in generating keys.

4. PUBLIC KEY (e)

e = 3

- This is the public key.
- It must be coprime with ϕ(n).

5. PRIVATE KEY (d)

d = 7

- This is the private key.
- It is calculated so that:

- $(e \times d) \% \phi(n) = 1$

6. ENCRYPTION FUNCTION

def encrypt(message):

- Converts message into encrypted numbers.

Steps inside:

- Convert character → number (ord)
- **Apply formula:**

cipher = (num^e) % n

- Store results in a list

7. DECRYPTION FUNCTION

def decrypt(cipher_list):

- Converts encrypted numbers back to text.

Steps inside:

- Apply formula:

plain = (num^d) % n

- Convert number → character (chr)
- Combine characters into message

8. TESTING

message = "HI"

- Message to encrypt

enc = encrypt(message)

- Encrypts message

dec = decrypt(enc)

- Decrypts back to original message

SECURITY ANALYSIS

1. WEAK PRIME NUMBERS

- You used:
  - p = 3, q = 11
- These are very small
- Easy to break using factorization

Real RSA uses:

- 2048-bit or 4096-bit primes

2. SMALL KEY SPACE

- n = 33 is extremely small
- Attacker can easily try all values

3. DETERMINISTIC ENCRYPTION

- Same character always produces same cipher
- Makes pattern analysis possible

4. ASCII LEVEL ENCRYPTION (WEAK)

- Each character encrypted separately
- Vulnerable to frequency analysis

5. NO PADDING USED

- Real RSA uses padding like:
  - OAEP (Optimal Asymmetric Encryption Padding)
- Without padding → insecure

6. EASY BRUTE FORCE

- Since values are small:
    - attacker can try all possibilities

Task 2: Create a Digital Signature [5]

Objective: Understand digital signatures

Tasks:

• Generate RSA key pair

• Create a hash of a message (e.g., using SHA256)

• Sign the hash using private key

• Verify it using the public key

• Modify the message slightly and show that verification fails

CODE:

```python
import hashlib

# ---------------- RSA KEY GENERATION ----------------
p = 3
q = 11

n = p * q
phi = (p - 1) * (q - 1)

e = 3
d = 7

print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))


# ---------------- CREATE HASH ----------------
def get_hash(message):  2 usages
    return hashlib.sha256(message.encode()).hexdigest()


# ---------------- SIGN MESSAGE ----------------
def sign(message):  1 usage
    hash_value = get_hash(message)
    hash_int = int(hash_value, 16)     # convert hex → integer
```

```python
    signature = pow(hash_int, d, n)    # encrypt hash with private key
    return signature


# ---------------- VERIFY SIGNATURE ----------------
def verify(message, signature):  2 usages
    new_hash = get_hash(message)
    new_hash_int = int(new_hash, 16)

    decrypted_hash = pow(signature, e, n)  # decrypt using public key

    return new_hash_int % n == decrypted_hash


# ---------------- TEST ----------------
message = "HELLO"

print("\nOriginal Message:", message)

signature = sign(message)
print("Signature:", signature)

result = verify(message, signature)
print("Verification (original):", result)
```

```python
# ---------------- MODIFIED MESSAGE TEST ----------------
fake_message = "HELLOO"   # changed message

result2 = verify(fake_message, signature)
print("Verification (modified message):", result2)
```

OUTPUT:

```
C:\Users\hashi\PycharmProjects\PythonProject\.venv\Scrip
Public Key (e, n): (3, 33)
Private Key (d, n): (7, 33)

Original Message: HELLO
Signature: 6
Verification (original): True
Verification (modified message): False

Process finished with exit code 0
```

LINE-BY-LINE SIMPLE EXPLANATION:

1. IMPORT LIBRARY

import hashlib

- Imports SHA-256 hashing function
- Used to create message fingerprint

2. RSA KEY GENERATION

p = 3
q = 11

- Two prime numbers used to generate RSA keys

n = p * q

- Multiply primes → n = 33
- Used in both encryption and decryption

phi = (p - 1) * (q - 1)

- Euler's totient function → phi = 20
- Helps in key generation

e = 3

- Public key exponent
- Used for verification

d = 7

- Private key exponent
- Used for signing

3. PRINT KEYS

```
print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))
```

- Shows public and private keys

4. CREATE HASH FUNCTION

```
def get_hash(message):
```

- Function to create SHA-256 hash

```
return hashlib.sha256(message.encode()).hexdigest()
```

- Converts message → hash value
- .hexdigest() gives hex string

5. SIGN FUNCTION

```
def sign(message):
```

- Creates digital signature

```
hash_value = get_hash(message)
```

- Get hash of message

hash_int = int(hash_value, 16)

- Convert hex hash → integer
- Needed for RSA math

signature = pow(hash_int, d, n)

- **RSA signing formula:**

signature = (hash^d) mod n

- Uses private key

return signature

- Returns digital signature

6. VERIFY FUNCTION

def verify(message, signature):

- Checks if message is valid

new_hash = get_hash(message)

- Recalculate hash of received message

new_hash_int = int(new_hash, 16)

- Convert hash to integer

decrypted_hash = pow(signature, e, n)

- **RSA verification formula:**

hash = (signature^e) mod n

- Uses public key

return new_hash_int % n == decrypted_hash

- Compare both hashes

- If equal → message is authentic

7. TEST MESSAGE

message = "HELLO"

- Original message

signature = sign(message)

- Generate digital signature

result = verify(message, signature)

- Check validity → should be True

8. MODIFIED MESSAGE TEST

fake_message = "HELLOO"

- Slightly changed message

result2 = verify(fake_message, signature)

- Verification fails because message changed

SECURITY ANALYSIS:

1. AUTHENTICITY (WHO SENT THE MESSAGE)

Digital signature ensures the sender is real.

- Only the sender has the private key (d)
- So only they can create the signature

If signature is valid → sender is verified

2. INTEGRITY (MESSAGE NOT CHANGED)

Ensures message is not modified during transmission

- Message is hashed using SHA-256

- Even 1 character change → hash completely changes

If message is changed:

- Hash mismatch happens
- Verification fails

3. NON-REPUDIATION (SENDER CANNOT DENY)

Sender cannot deny sending message

- Because signature is created using private key
- Only sender owns private key

So sender is legally responsible

4. USE OF HASHING (SHA-256 SECURITY)

SHA-256 is strong and secure

- Produces fixed-size output
- Very difficult to reverse
- No two different messages produce same hash (practically)

Prevents message tampering

5. RSA SECURITY (KEY BASED PROTECTION)

Security depends on RSA algorithm

- Public key is known to everyone
- Private key is secret

Attacker cannot easily compute private key if RSA is strong